

PalmSource®

Conference and Expo

PalmSource®
Conference and Expo



Effective C++ Development Using CodeWarrior® Tools

Ben Combee, Palm OS® Tools Technical Lead



C++ is a great language for implementing Palm OS® applications, and CodeWarrior™ tools provide the best environment for using C++.



Agenda Slide

C++: What Overhead?

Exception Handling

Templates

Expanded Mode

Object Library for Palm OS

metrowerks



C++: The Big Advantages

- More robust programs
  - Data encapsulation
  - Constructors and destructors
  - Better type safety
- Program models real-life objects
- Easier to rework and reuse code
- Classes provide natural organization
- Widely used in desktop programming



C++: No Overhead

- MYTH: C++ makes your application larger
- This is wrong – just compiling your application as C++ won't add any bulk
- Namespaces, function overloading, type checking, and simple classes don't add bulk

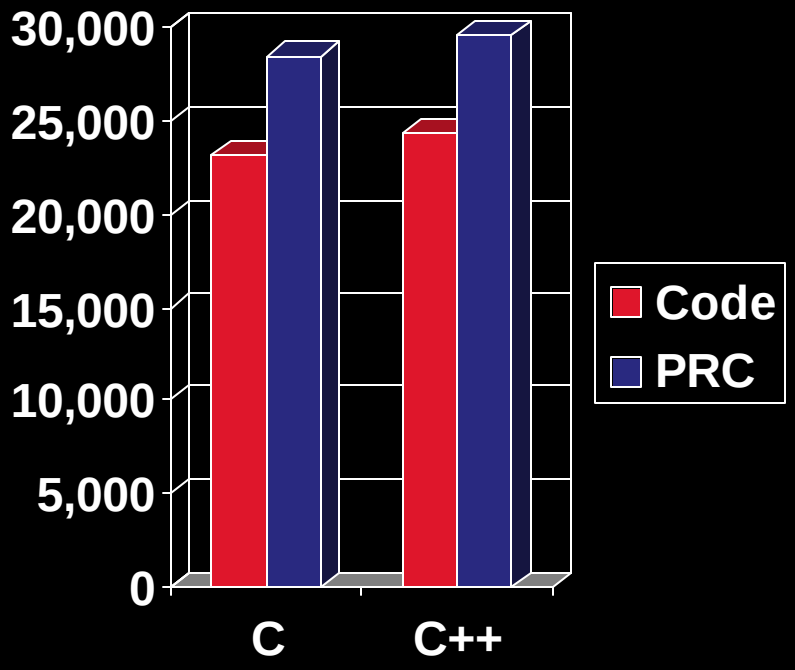


C++: Some Overhead

- C++ can add bulk when used carelessly
- Some overhead comes from the runtime library
- Constructors and destructors add hidden code
- Template instantiations can add up
- Exception handling adds a cost per function call

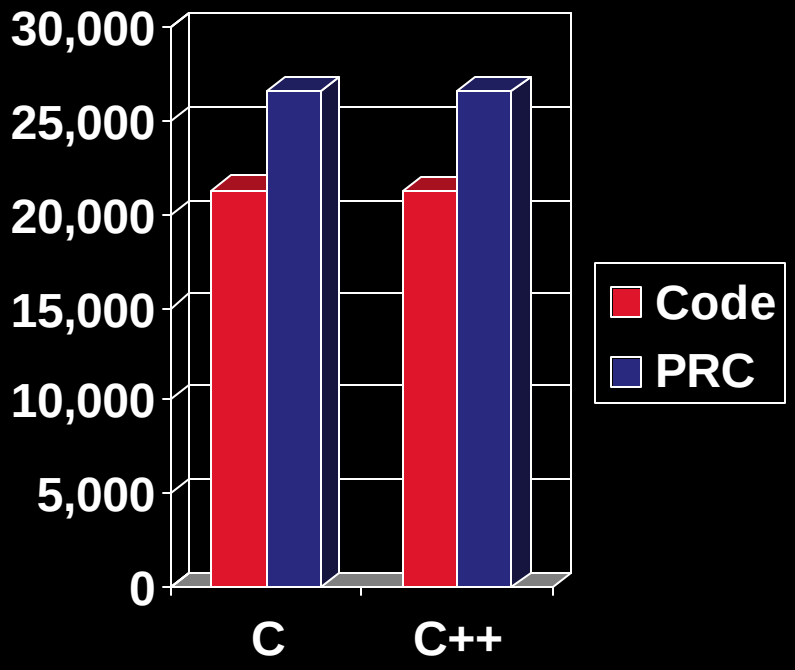
C++ Overhead: The Numbers

- MemoPad.mcp from Palm OS 4.0 SDK Examples
- CW Palm OS V8.1
- Had to fix 84 errors, almost all void pointer conversions when compiling source as C++
- Uses StartupCode.lib which doesn't support advanced C++ features



C++ Overhead: More Numbers

- MemoPad.mcp from Palm OS 4.0 SDK Examples
- CW Palm OS V8.1
- MacsBug debug symbols disabled to reduce the code size
- Dropping the symbols reduced code size 20%





C++: One-Time Overhead

- Using the CodeWarrior C++ runtime library increased the PRC size to 31KB, a 2K increase
- Extra size is due to extra capabilities:
  - Multiple code segments
  - longjmp() and abort() support
  - Exception handling support
  - new and delete implementations
  - Static initializers and global destructor chain



Agenda Slide

- C++: What Overhead?
- Exception Handling
- Templates
- Expanded Mode
- Object Library for Palm OS

metrowerks



What Does Exception Handling Buy Me?

- Exception handling is a structured method for indicating errors or unusual situations
- Avoids need for lots of errors checking code
- Allows more detailed information to be returned

```
try { C++ code }
catch (CException &e) { C++ code }
catch (...) { C++ code }
CException e; throw e;
```



Exceptions and Palm OS

- Many developers who started on Windows have a wrong impression of C++ exceptions
- On Win32, a **`catch(...)`** also catches system errors like NULL pointer references
- This behavior is outside the C++ standard
- On Palm OS, C++ won't catch OS errors or hardware exceptions, only thrown ones
- Exceptions can't be thrown across OS code



What Does Exception Handling Cost?

- CodeWarrior implements zero-overhead exception handling
- This refers to run-time overhead
- Exceptions require compiler-generated tables
- Runtime library uses the tables to route exceptions to the proper handler
- Table size depends on exception use and number of objects with destructors



Cost of Exceptions: Ideal Numbers

- MemoPad.mcp from Palm OS 4.0 SDK
- CW for Palm OS V8
- Turning on exceptions added 160 bytes of data
- This is minimum overhead
  - MemoPad doesn't use exceptions
  - Cost of 4 bytes per function in projects



Cost of Exceptions: Real World Numbers

- Expect a 15:1 code-to-data ratio for exceptions
  - 150K of code = 10K of exception tables
- Additional code growth for exception handlers
- Code size often eclipsed by savings from avoiding redundant error checking code
- Use inline function wrapper or macros to automate turning OS errors into exceptions



Reducing Exception Overhead

- Throw specifiers let you indicate what will be thrown from a function
- void f() throw () means nothing will be thrown
- Minimize use of objects with destructors
- Destructors of live objects get called until the exception is caught, so objects add to table size
- Be sure to turn on compiler optimizations



Header Hacks to Reduce Overhead

- Metrowerks is not allowed to change Palm's SDK header files
- However, there is a small change you can make that we just discovered a few weeks ago
- In PalmTraps.h, modify the OS_CALL macros to add "**`throw ()`**" to the prototypes
- This will reduce exception table size by 5 to 20%
- Works by telling C++ compiler that Palm OS routines don't throw exceptions



Agenda Slide

- C++: What Overhead?
- Exception Handling
- Templates
- Expanded Mode
- Object Library for Palm OS



What are templates?

- A template is a C++ definition with placeholders
- Template arguments can by types, integers, or pointers to global data items
- Used by STL (Standard Template Library)
- Useful for containers, generic algorithms
- Must be careful on devices with limited code size, for example, Palm OS!



Templates: Simple Example

```
template <class T>
class Pair {
private:
    T m_item1, m_item2;
public:
    Pair(const T& t1, const T& t2); // constructor
    T& part1() { return m_item1; }
    T& part2() { return m_item2; }
};

Pair<UInt16> coordinate;
```



Templates: Uses on Palm OS

- Handling locking and unlocking of MemHandles
- Freeing pointers when they are unused
- Customized container classes
- Implementing generic algorithms



Template Example: AutoMemHandle

```
template <class T>
class AutoMemHandle {
private: T* mPtr;
public:
  AutoMemHandle(MemHandle mh)
    { mPtr = (T*)MemHandleLock(mh); }
  ~AutoMemHandle()
    { MemPtrUnlock(mPtr); }
  operator typename T*()
    { return mPtr; } };
```



AutoMemHandle continued

- That's just the start of the definition
- Needs to handle dereference (->) and indirect (*) operators, and returning reference types
- Also needs to define copy semantics
- Having a pointer into the interior of a handle can also be useful
- When properly implemented, can be used to pass locked handles between functions without unneeded locking/unlocking



Templates: Impact on Code Size

- On template use, the compiler generates a new instance of the function or class
- Code generated as “link-once”
- “link-once” code forced into first segment
- Careful inlining can reduce size impact
- Can control location by explicit instantiation
- For example:

```
template class Pair<Int16>;
```



Agenda Slide

C++: What Overhead?

Exception Handling

Templates

Expanded Mode

Object Library for Palm OS



What is Expanded Mode?

- New code generation method in V8
- C and C++ use several data structures
  - Virtual function tables
  - Run time type information
  - Exception handling tables
  - Cross-segment jump tables
- Expanded mode moves these from dynamic heap into storage heap
- Requires Palm OS 3.0



Expanded Mode Advantages

- More free dynamic heap available for global variables, MemPtrNew, MemHandleNew
- Read-only structures protected from corruption
- Can use C++ classes with virtual functions during “no global variable” launch codes
- ZooKeeper example from Bear River
  - non-expanded: 8790 bytes of global data
  - expanded mode: 5084 bytes of global data

Expanded Mode Disadvantages

- Code rework to use “thunks” for OS callbacks
  - needed since expanded mode uses a 68K register that the OS might trash
- PRC file size increases
  - additional size of runtime library
  - code generation is less efficient due to fewer registers
- ZooKeeper code size
  - Non-expanded mode: 92 kilobytes
  - Expanded-mode: 94 kilobytes



More Information on Expanded Mode

- CodeWarrior white paper on Expanded Mode
  - http://www.metrowerks.com/products/palm/?whitepapers
- Scott Maxwell's macros to handle thunks
  - Included in CW Palm OS 8.2 patch
- Download the CW Palm OS 8.1 patch
  - http://www.metrowerks.com/download
  - Contains major runtime library fixes



Agenda Slide

C++: What Overhead?

Exception Handling

Templates

Expanded Mode

Object Library for Palm OS



What is a C++ Framework?

- A framework is a collection of classes that provide a skeleton for a standard application
- Your application is a specialization of the framework with some behaviors overridden
- Can reduce time required to build a new program and time needed to learn a system
- Examples include MFC, PowerPlant, Qt, GTK+



Object Library for Palm OS

- Modeled on MFC
  - Light wrappers for system objects
  - Event handling implemented by mapping macros
  - Small set of utility classes for strings, containers, and memory management
- Designed to work with CodeWarrior
- Split into two static libraries
  - Essential classes are linked into segment 1
  - UI classes are part of segment 2



CodeWarrior and POL

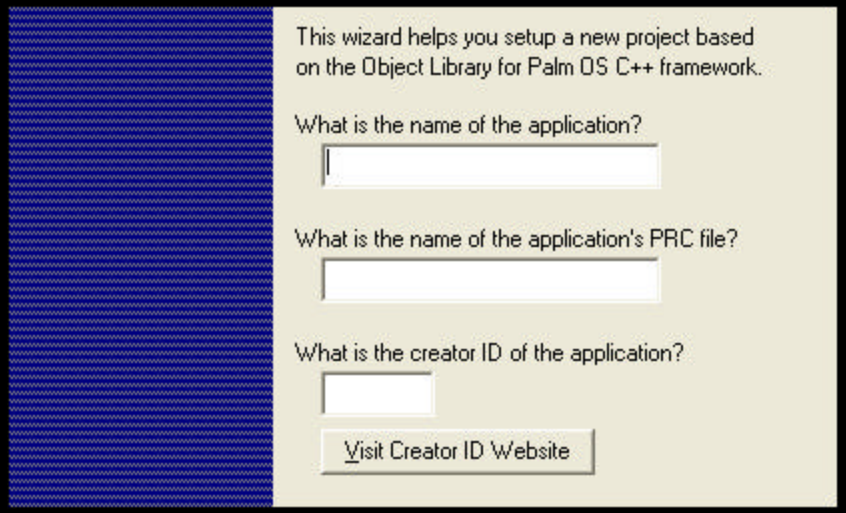
- Referred to as “POL” for historical reasons
- Metrowerks has licensed this library for inclusion with CodeWarrior tools for Palm OS
- New support being developed
  - Integrate POL documentation into IDE
  - Add POL Application wizard for new projects
- Resource editing improvements in V9 will include POL-based code generation

Using POL in an Application

- Demonstration using Metrowerks Survey application
  - Shows use of UI classes
  - Also uses record streaming to add new entries to the internal database
  - Overrides default application logic to set a procedural alarm to reset app after five minutes
  - Templates used to implement similar forms

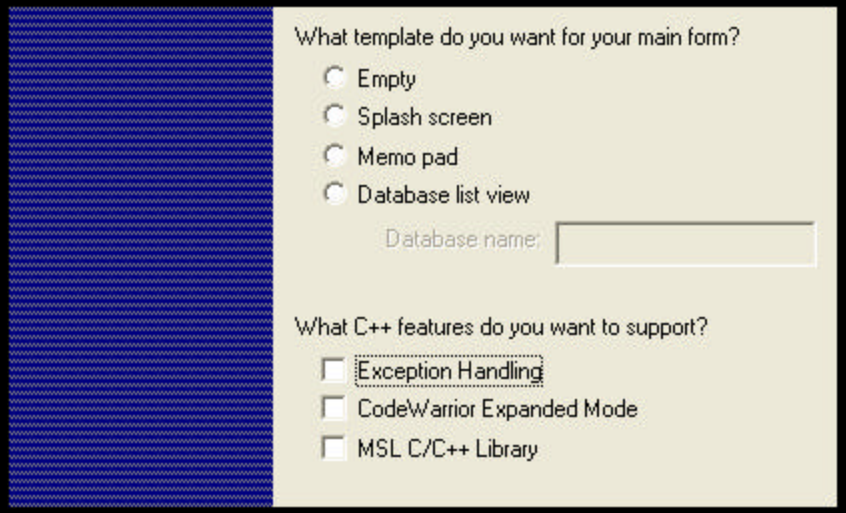
New Object Library Application Wizard

- Not yet completed to show in CodeWarrior IDE
- Here are the wizard dialog boxes, shown out of context:



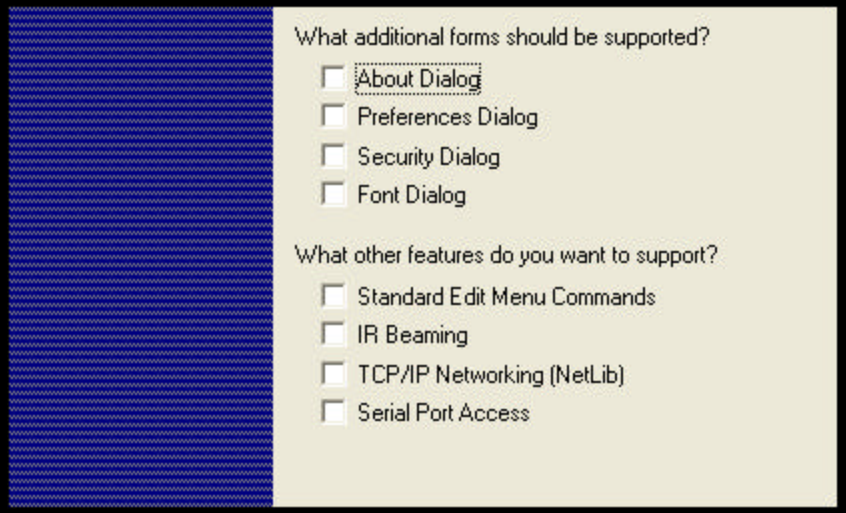
New Object Library Application Wizard

- Not yet completed to show in CodeWarrior IDE
- Here are some of the wizard dialog boxes, shown out of context:



New Object Library Application Wizard

- Not yet completed to show in CodeWarrior IDE
- Here are some of the wizard dialog boxes, shown out of context:





POL Application Structure

- Use DEFAULT_STARTER macro
- Applications inherit from CPalmApp class
- Macros map frmLoad events to an instantiation of a CForm-derived class
- Form classes define an event map that converts Palm OS events into method calls
- Can use controls directly or through wrapper classes



POL and Palm OS Databases

- CDatabase and CRecord classes wrap the Data Manager APIs
- CFile wraps the File Stream APIs
- CDBListCtrl and CDBGrid classes support hooking databases directly to user interface
- No direct hooks into database systems like Oracle 8i Lite, iAnywhere, or Palm Identicon



Advanced POL Usage

- TCP/IP (NetLib)
- Exchange Manager (IrDA, BlueTooth)
- Serial communication classes
- Advanced Controls
  - CMemo – multi-line edit control with scroll bar
  - CTreeView – hierarchical list view
  - Custom control framework
- Bitmap animation
- Buffered input/output streams



PalmSource®

Conference and Expo